

Leva.Framework.Sample.WebIdentity

Web sample for Identity.AspNet JWT bearer authentication and optional Google OAuth sign-in. It keeps advanced web identity flows separate from LocalIdentity so local cookie/session authentication stays simple.

```
Leva.Framework.Sample.WebIdentity
  -> Leva.Framework.Core
  -> Leva.Framework.Identity
  -> Leva.Framework.Identity.Memory
  -> Leva.Framework.Identity.Local
  -> Leva.Framework.Identity.AspNet
  -> Microsoft.AspNetCore
```

Setup shown by the sample

```
Program.cs
  -> creates local principals used for JWT login
  -> creates AuthenticationService with LocalAuthenticationPolicy
  -> registers Identity.AspNet JWT services
  -> maps JWT login/token endpoints
  -> maps a bearer-token protected API endpoint
  -> registers optional Google sign-in services when credentials are configured

JWT setup
  -> AddAspNetJwtPrincipalServices(...)
  -> MapAspNetJwtEndpoints()
  -> RequireAuthorization("JwtApiUse") on /api/jwt/admin-area

Google setup
  -> AddAspNetGooglePrincipalServices(...) only when client id/secret exist
  -> MapAspNetGoogleEndpoints() only when Google is configured
  -> /sample/google/status tells the browser whether Google is ready
  -> /sample/google/principal shows the current signed-in browser user
```

JWT behavior shown

```
local credentials      -> JWT token
  missing token        -> rejected with 401
  invalid token        -> rejected with 401
  valid token          -> protected API succeeds
  wrong password       -> rejected with 401
```

JWT works without external setup.

Google behavior shown

```
Google not configured -> page shows setup instructions
  Google configured   -> page exposes sign-in flow
  Google callback      -> page can show the signed-in browser user
```

Google OAuth requires external credentials, so this part is optional. Basic Google OAuth sign-in with profile/email scopes usually has no direct Google OAuth fee, but public production apps can still be subject to Google consent-screen verification, quota limits, and policy requirements.

Google OAuth setup

Use these names for the Google setup:

```
Google Cloud project name: LevaFrameworkWebIdentity
  OAuth app name:          Leva Framework Web Identity
  OAuth client name:       Leva Framework Web Identity - Localhost
```

1. Open Google Cloud Console.
2. Create or select the `LevaFrameworkWebIdentity` project.
3. Configure the OAuth consent screen / Google Auth Platform branding:
 - App name: `Leva Framework Web Identity`
 - User support email: your email
 - Developer contact email: your email
4. Create an OAuth client:
 - Application type: Web application
 - Name: `Leva Framework Web Identity - Localhost`
 - Authorized redirect URI for default `dotnet run`: `http://localhost:5000/identity/google/callback`

- Authorized redirect URI for `samples/run-all.sh`: `http://localhost:5105/identity/google/callback`

5. Store the client credentials outside source control.

Use user secrets:

```
dotnet user-secrets set "Google:ClientId" "<client-id>" --project samples/Leva.Framework.Sample.WebIdentity
dotnet user-secrets set "Google:ClientSecret" "<client-secret>" --project samples/Leva.Framework.Sample.WebIdentity
```

Or use .NET configuration environment variables:

```
export Google__ClientId="<client-id>"
export Google__ClientSecret="<client-secret>"
```

The sample also accepts these explicit environment variable names:

```
export LEVA_GOOGLE_CLIENT_ID="<client-id>"
export LEVA_GOOGLE_CLIENT_SECRET="<client-secret>"
```

When both values are configured, the page enables the Google sign-in button. Without them, the page shows Google as not configured instead of failing. The page also prints the callback URL for the currently running host; that printed callback URL is the source of truth if you run on a different port. After the callback succeeds, **Refresh Google sign-in** shows the signed-in browser user.

Run and expected result

```
dotnet run --project samples/Leva.Framework.Sample.WebIdentity
```

```
PASS: missing bearer token is rejected
PASS: invalid bearer token is rejected
PASS: local credential login issues JWT
PASS: valid JWT can access protected API
PASS: wrong password cannot issue JWT
Google OAuth: configured only after client id/secret are provided
Google sign-in: signed-in browser user is visible after callback
```